

Lab 2.1: Libraries, Linking, and ELF Executable Structure

1. Introduction

This laboratory exercise focuses on understanding how a C program uses libraries, how programs are linked, and how Linux stores and loads executable files.

When a C program uses functions such as `printf()`, `getpid()`, `getppid()`, and `time()`, these functions are provided by the C Standard Library. The compiler uses header files to understand these functions, while the linker connects the program with the actual library code.

This lab also investigates the difference between static and dynamic linking and uses `readelf` and `ldd` to examine ELF executable files and their library dependencies.

2. Objectives

The main objectives of this laboratory were:

To create and compile a practical C program using library functions.

To understand the purpose of C header files.

To locate C library files on a Linux system.

To understand static and dynamic linking.

To compare static and dynamically linked executables.

To understand the ELF executable format.

To inspect ELF headers and program headers using readelf.

To identify dynamic library dependencies using readelf and ldd.

3. Creating the C Program

A C source file named procinfo.c was then created.

Program Code

```
GNU nano 8.7.1 procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {

    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");

    return 0;
}
```

4. Compiling and Running the Program

The gcc compiler converts the C source code into an executable program. The program was compiled using:

```
root@AYUSHA:~/lab2# nano procinfo.c
root@AYUSHA:~/lab2# gcc procinfo.c -o procinfo
```

The executable was then run using:

```
root@AYUSHA:~/lab2# ls
procinfo  procinfo.c
root@AYUSHA:~/lab2# ./procinfo
=== Process Information ===
My Process ID (PID): 587
Parent Process ID: 337
Current Time: Sun Oct  4 10:04:46 2026
Executable Path: /proc/self/exe
root@AYUSHA:~/lab2#
```

The PID and PPID are generated by the operating system and may be different each time the program is executed. The compilation and execution demonstrate that the C program successfully uses functions provided by the C library.

5. C Header Files

Header files contain declarations, function prototypes, macros, and type definitions. They tell the compiler how functions can be used, but they do not contain the complete executable machine code of those functions.

The command displayed information from the `stdio.h` header. The location of the header files was checked using:

```
root@AYUSHA:~/lab2# ls /usr/include/stdio.h
/usr/include/stdio.h
root@AYUSHA:~/lab2# ls /usr/include/unistd.h
/usr/include/unistd.h
```

6. Checking Function Declarations

Similarly, the declaration of `getpid()` was checked using:

```
root@AYUSHA:~/lab2# grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN. */
extern int fprintf (FILE *__restrict __stream,
```

This demonstrates that the header file provides the compiler with information about how the function is declared. The actual compiled implementation of the function is stored in the C library.

7. Locating the C Libraries

The shared C library and the static C library was checked using:

```
-bash: /lib/x86_64-linux-gnu/: Is a directory
root@AYUSHA:~/lab2# /usr/lib/x86_64-linux-gnu/
-bash: /usr/lib/x86_64-linux-gnu/: Is a directory
root@AYUSHA:~/lab2# /usr/lib/
-bash: /usr/lib/: Is a directory
root@AYUSHA:~/lab2# /lib/
-bash: /lib/: Is a directory
root@AYUSHA:~/lab2# grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid (void) __THROW;
root@AYUSHA:~/lab2# ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so  /lib/x86_64-linux-gnu/libc.so.6
root@AYUSHA:~/lab2# ls /usr/lib/x86_64-linux-gnu/libc.a
/usr/lib/x86_64-linux-gnu/libc.a
```

The C Standard Library is commonly provided by glibc on Linux.

A static library contains compiled object code that can be included directly inside an executable during static linking.

A shared library is kept separately from the executable and can be loaded when the program runs.

8. Static Linking

A statically linked version of the program was created using:

```
root@AYUSHA:~/lab2# gcc -static procinfo.c -o procinfo_static
root@AYUSHA:~/lab2# gcc -static procinfo.c -o procinfo_static
root@AYUSHA:~/lab2# ls -lh procinfo_static
```

The size of the executable was checked using:

```
root@AYUSHA:~/lab2# ls -lh procinfo_static
-rwxr-xr-x 1 root root 961K Oct  4 10:32 procinfo_static
```

In static linking, the required library code is included directly in the executable. Therefore, the executable becomes larger but has fewer runtime library dependencies.

9. Dynamic Linking

A dynamically linked executable was created using the below command and Its size was checked using:

```
root@AYUSHA:~/lab2# gcc procinfo.c -o procinfo_dynamic
root@AYUSHA:~/lab2# ls -lh procinfo_dynamic
-rwxr-xr-x 1 root root 16K Oct  4 10:35 procinfo_dynamic
```

In dynamic linking, the executable does not contain all of the required library code. Instead, it contains references to shared libraries such as libc.so.6.

Dynamic linking generally produces smaller executables and allows multiple programs to share the same library in memory.

10. ELF Executable Format

Linux compiled executables use the ELF format.

ELF stands for: **Executable and Linkable Format**

The readelf command was used to inspect the internal structure of the executable without executing it.

There are three important views examined in this lab:

1. ELF Header
2. Program Headers
3. Dynamic Section

11. Inspecting the ELF Header

The ELF header was inspected using:

```
root@AYUSHA:~/lab2# readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                               2's complement, little endian
  Version:                             1 (current)
  OS/ABI:                              UNIX - System V
  ABI Version:                         0
  Type:                                DYN (Position-Independent Executable file)
  Machine:                             Advanced Micro Devices X86-64
  Version:                             0x1
  Entry point address:                 0x1140
  Start of program headers:            64 (bytes into file)
  Start of section headers:           14264 (bytes into file)
  Flags:                               0x0
  Size of this header:                 64 (bytes)
  Size of program headers:             56 (bytes)
  Number of program headers:           14
  Size of section headers:             64 (bytes)
  Number of section headers:           31
  Section header string table index: 30
root@AYUSHA:~/lab2# readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 14 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz              MemSiz              Flags  Align
PHDR             0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000310 0x0000000000000310  R      0x8
INTERP          0x0000000000000374 0x0000000000000374 0x0000000000000374
                 0x000000000000001c 0x000000000000001c  R      0x1
    [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD            0x0000000000000000 0x0000000000000000 0x0000000000000000
```

The ELF header provides general information about the executable.

Important information includes:

- File type
- CPU architecture
- 32-bit or 64-bit class
- Entry point address

12. Inspecting Program Headers

Program headers were inspected using:

```
root@AYUSHA:~/lab2# readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 14 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz             MemSiz             Flags   Align
PHDR             0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000310 0x0000000000000310 R       0x8
INTERP           0x0000000000000374 0x0000000000000374 0x0000000000000374
                 0x000000000000001c 0x000000000000001c R       0x1
    [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD             0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x000000000000007b8 0x000000000000007b8 R       0x1000
LOAD             0x00000000000001000 0x00000000000001000 0x00000000000001000
                 0x00000000000000311 0x00000000000000311 R E     0x1000
LOAD             0x00000000000002000 0x00000000000002000 0x00000000000002000
                 0x000000000000001c0 0x000000000000001c0 R       0x1000
LOAD             0x00000000000002d80 0x00000000000003d80 0x00000000000003d80
                 0x00000000000000290 0x00000000000000298 RW      0x1000
DYNAMIC          0x00000000000002d90 0x00000000000003d90 0x00000000000003d90
                 0x000000000000001f0 0x000000000000001f0 RW      0x8
NOTE            0x00000000000000350 0x00000000000000350 0x00000000000000350
                 0x00000000000000024 0x00000000000000024 R       0x4
NOTE            0x00000000000002170 0x00000000000002170 0x00000000000002170
                 0x00000000000000030 0x00000000000000030 R       0x8
NOTE            0x000000000000021a0 0x000000000000021a0 0x000000000000021a0
                 0x00000000000000020 0x00000000000000020 R       0x4
GNU_PROPERTY    0x00000000000002170 0x00000000000002170 0x00000000000002170
                 0x00000000000000030 0x00000000000000030 R       0x8
GNU_EH_FRAME    0x00000000000002088 0x00000000000002088 0x00000000000002088
                 0x00000000000000034 0x00000000000000034 R       0x4
GNU_STACK       0x00000000000000000 0x00000000000000000 0x00000000000000000
                 0x00000000000000000 0x00000000000000000 RW      0x10
GNU_RELRO       0x00000000000002d80 0x00000000000003d80 0x00000000000003d80
                 0x00000000000000280 0x00000000000000280 R       0x1
```

Program headers describe the segments that the operating system loads into virtual memory. INTERP identifies the dynamic linker that is required to load shared libraries. LOAD segments describe regions of the executable that are loaded into memory. GNU_STACK provides information about stack permissions.

13. Inspecting the Dynamic Section

It identifies shared libraries required by the executable. This means that the dynamically linked executable requires the C Standard Library at runtime. The dynamic section was inspected using:

```
root@AYUSHA:~/lab2# readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
   Tag               Type              Name/Value
0x0000000000000001 (NEEDED)         Shared library: [libc.so.6]
0x000000000000000c (INIT)           0x1000
0x000000000000000d (FINI)           0x1304
0x0000000000000019 (INIT_ARRAY)     0x3d80
0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
0x000000000000001a (FINI_ARRAY)     0x3d88
0x000000000000001c (FINI_ARRAYSZ)    8 (bytes)
0x0000000006ffffef5 (GNU_HASH)       0x390
0x0000000000000005 (STRTAB)         0x508
0x0000000000000006 (SYMTAB)         0x3b8
0x000000000000000a (STRSZ)          208 (bytes)
0x000000000000000b (SYMENT)         24 (bytes)
0x0000000000000015 (DEBUG)          0x0
0x0000000000000003 (PLTGOT)         0x3f80
0x0000000000000002 (PLTRELSZ)       192 (bytes)
0x0000000000000014 (PLTREL)         RELA
0x0000000000000017 (JMPREL)         0x6f8
0x0000000000000007 (RELA)           0x638
0x0000000000000008 (RELASZ)         192 (bytes)
0x0000000000000009 (RELAENT)        24 (bytes)
0x000000000000001e (FLAGS)          BIND_NOW
0x0000000006ffffffb (FLAGS_1)       Flags: NOW PIE
0x0000000006ffffffe (VERNEED)       0x5f8
0x0000000006fffffff (VERNEEDNUM)    1
0x0000000006ffffff0 (VERSYM)        0x5d8
0x0000000006ffffff9 (RELACOUNT)     3
0x0000000000000000 (NULL)           0x0
root@AYUSHA:~/lab2# ldd procinfo_dynamic
        linux-vdso.so.1 (0x00007105236e8000)
        libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x0000710523400000)
)
        /lib64/ld-linux-x86-64.so.2 (0x00007105236ea000)
```



```

root@AYUSHA:~/lab2# readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
  Tag                Type                Name/Value
0x0000000000000001 (NEEDED)           Shared library: [libc.so.6]
0x000000000000000c (INIT)             0x1000
0x000000000000000d (FINI)             0x1304
0x0000000000000019 (INIT_ARRAY)       0x3d80
0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
0x000000000000001a (FINI_ARRAY)       0x3d88
0x000000000000001c (FINI_ARRAYSZ)      8 (bytes)
0x000000006ffffef5 (GNU_HASH)           0x390
0x0000000000000005 (STRTAB)           0x508
0x0000000000000006 (SYMTAB)           0x3b8
0x000000000000000a (STRSZ)            208 (bytes)
0x000000000000000b (SYMENT)           24 (bytes)
0x0000000000000015 (DEBUG)            0x0
0x0000000000000003 (PLTGOT)           0x3f80
0x0000000000000002 (PLTRELSZ)         192 (bytes)
0x0000000000000014 (PLTREL)           RELA
0x0000000000000017 (JMPREL)           0x6f8
0x0000000000000007 (RELA)             0x638
0x0000000000000008 (RELASZ)           192 (bytes)
0x0000000000000009 (RELAENT)          24 (bytes)
0x000000000000001e (FLAGS)            BIND_NOW
0x000000006fffffbb (FLAGS_1)          Flags: NOW PIE
0x000000006ffffffe (VERNEED)          0x5f8
0x000000006fffffff (VERNEEDNUM)       1
0x000000006fffffff0 (VERSYM)          0x5d8
0x000000006fffffff9 (RELACOUNT)       3
0x0000000000000000 (NULL)             0x0
root@AYUSHA:~/lab2# ldd procinfo_dynamic
        linux-vdso.so.1 (0x00007f6b1d338000)
        libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007f6b1d000000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f6b1d33a000)
root@AYUSHA:~/lab2# ldd procinfo_static
        not a dynamic executable

```

16. Knowledge Check

1. What is the fundamental difference between a C header file and a shared library?

A header file contains declarations and information about functions, while a shared library contains the compiled machine code that can be loaded and used at runtime.

2. What is static and dynamic linking?

Static linking includes the required library code inside the executable. Dynamic linking keeps the library separate and loads it when the program runs.

3. What is an ELF Header?

The ELF Header contains important information about an ELF executable, such as its architecture, file type, and entry point.

4. What does a bookmark mean?

A bookmark is a saved reference or location used to quickly return to a particular page, section, or resource.

5. What was the main finding of this laboratory?

The main finding was that a C program depends on libraries during compilation and execution. Linux uses ELF executable information, a dynamic linker, and shared libraries to load and run dynamically linked programs.

The main idea is to understand **what happens between writing C code and running the program**

"Take my C program, compile and link it, and create an executable called `procinfo`."

The `.h` file mainly contains **information telling the compiler what a function looks like**.

Because the lab wants you to physically see where the C header files are located.

Make the `dynamic version` explicitly